CO1 ASSESSMENT 1

C Programming Assignment

Student Name	Vignesh Kumar K
Register Number	192512375
Course Code & Name	CSA0208, C programming
Slot	A
Assignment / Task Title	Reliable Billing Program for a Rural Electricity Co-operative
GitHub Repository URL	https://github.com/192512375/CSA02/tree/main/Assessment%201
Submission Date	27-10-2026

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

TASK 1 — Understand the Problem and Identify the Data

Task / Problem Statement

Grama Jyothi Rural Electricity Co-operative bills about 4,000 households every month. The program must calculate each household's electricity consumption and bill using the previous and current meter readings, tariff slabs, and fixed charge. It must handle 16-bit meter rollover and calculate the bill accurately to the paise.

Aim

To identify the inputs, outputs, constants, rules, and assumptions required for a reliable electricity billing program in C.

Concept / Theory

Inputs are household number, consumer category, previous meter reading, and current meter reading. Outputs are units consumed, energy charge, fixed charge, and total bill. The tariff values are: first 100 units free, units 101–200 at 225 paise per unit, units above 200 at 450 paise per unit, and a fixed charge of 5,000 paise. The meter register ranges from 0 to 65,535. Money should be stored in integer paise to avoid floating-point rounding errors. The bill must not be negative, and total bill must equal energy charge plus fixed charge. Assumption: no taxes, subsidies, penalties, or extra fees are included.

Procedure / Algorithm

1. Read the household number, category, previous reading, and current reading.

2. Calculate units consumed, accounting for meter rollover.

3. Calculate the energy charge using the tariff slabs.

4. Add the fixed charge in paise.

5. Display the total bill in rupees and paise and validate the result.

Program / Source Code

```
/* Task 1: Data identification only; implementation is completed in Tasks 2 and 3. */
#include <stdio.h>

int main(void)
{
    unsigned short previous_reading = 1200;
    unsigned short current_reading = 1530;
    unsigned int units = current_reading - previous_reading;

    printf("Previous reading: %hu\n", previous_reading);
    printf("Current reading: %hu\n", current_reading);
    printf("Units consumed: %u\n", units);
    return 0;
}
```

Output Screenshot

Expected output:

Previous reading: 1200
Current reading: 1530
Units consumed: 330

Analysis / Discussion

The inputs and outputs have been identified. Meter readings use integer values; monetary values should be stored as paise. If readings were stored as text such as "01530", conversion would take O(d) time for d digits and the text would use O(d) space. Fixed-width integer reading and comparison take O(1) time and O(1) space.

Result

The billing problem's required data, constants, rules, and assumptions have been identified.

TASK 2 — Choose the Data Types

Task / Problem Statement

Choose suitable C data types for meter readings and consumption. Complete the rollover calculation so that a 16-bit meter resetting from 65,535 to 0 does not produce a negative result.

Aim

To select data types based on range and accuracy and implement rollover-safe meter consumption.

Concept / Theory

```
Use unsigned short for a 16-bit meter reading, unsigned int for consumption, size_t for sizes and counts, long long for money in paise, and
```

Procedure / Algorithm

1. Define meter_t as unsigned short.

2. Calculate the difference using arithmetic modulo 65,536.

3. Print the meter type size and maximum reading.

4. Test normal readings 1200 to 1530.

5. Test rollover readings 65400 to 120.

Program / Source Code

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    unsigned int modulus = (unsigned int)USHRT_MAX + 1U;
    return ((unsigned int)current + modulus - (unsigned int)previous)
           % modulus;
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
    return 0;
}
```

Output Screenshot

Expected output:

sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units

Analysis / Discussion

For the rollover case, consumption is (65,535 – 65,400) + 1 + 120 = 256 units. Each calculation is O(1) time and uses O(1) extra space. A floating-point total can accumulate representation error; integer paise represents each paisa exactly. Confirm the type size and USHRT_MAX on the compiler used.

Result

The selected data types support the required meter range and exact currency calculations. The rollover test returns 256 units.

TASK 3 — Define the Constants

Task / Problem Statement

Define tariff values and slab limits in one place, then calculate the bill in integer paise using the tariff slabs.

Aim

To implement a billing function that uses centralized tariff macros and calculates bills exactly to the paise.

Concept / Theory

```
The first 100 units are free, units 101–200 cost 225 paise each, and units above 200 cost 450 paise each. The fixed charge is 5,000 paise.
```

Procedure / Algorithm

1. Initialize the bill to the fixed charge.

2. Add 450 paise for each unit above 200.

3. Add 225 paise for each unit between 101 and 200.

4. Do not add an energy charge for the first 100 units.

5. Return the total in paise and display it as rupees and paise.

Program / Source Code

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000LL

long long compute_bill(int units)
{
    long long paise = FIXED_CHARGE_PAISE;

    if (units < 0)
        return 0;

    if (units > SLAB2_UNITS) {
```

```
        paise += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS)
        paise += (long long)(units - SLAB1_UNITS) * RATE2_PAISE;

    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = (int)(sizeof tests / sizeof tests[0]);

    for (int i = 0; i < n; i++) {
        long long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %lld.%02lld\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Output Screenshot

Expected output:

```
0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

Analysis / Discussion

The results match the specified slab rates. Changing RATE2_PAISE from 225 to 230 makes the 150-unit bill Rs 165.00 and the 200-unit bill Rs 280.00. Bill calculation takes O(1) time and O(1) extra space because the number of slabs is fixed. A typed const variable offers type checking and debugger visibility; runtime complexity is unchanged.

Result

The billing function calculates the expected totals and tariff changes can be made in one place.

TASK 4 — Use Macros Safely

Task / Problem Statement

Write macros for repeated calculations, including squaring a value and finding the larger of two values. Explain macro expansion, parentheses, repeated argument evaluation, and when a function is safer.

Aim

```
To use fully parenthesized macros and compare them with static inline functions.
```

Concept / Theory

Macros are expanded by the preprocessor before compilation. Parentheses around parameters and the complete expression prevent common precedence errors. A macro such as MAX(a, b) may evaluate an argument more than once, so expressions with side effects such as i++ should not be passed to it. Static inline functions provide type checking and evaluate each argument once.

Procedure / Algorithm

1. Define fully parenthesized SQUARE and MAX macros.

2. Define a macro to convert rupees to paise.

```
3. Define equivalent static inline functions for square and maximum.
```

4. Run sample inputs and compare the outputs.

5. Use gcc -E to inspect the expanded source.

Program / Source Code

```
#include <stdio.h>

#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
#define RUPEES_TO_PAISE(rupees) ((long long)(rupees) * 100LL)

static inline int square_int(int x)
{
    return x * x;
}

static inline int max_int(int a, int b)
{
    return (a > b) ? a : b;
}

int main(void)
{
    int a = 5;
    int b = 8;
```

```
    printf("SQUARE(a + 1) = %d\n", SQUARE(a + 1));
    printf("MAX(a, b) = %d\n", MAX(a, b));
    printf("100 rupees = %lld paise\n", RUPEES_TO_PAISE(100));
    printf("inline square_int(a + 1) = %d\n", square_int(a + 1));
    printf("inline max_int(a, b) = %d\n", max_int(a, b));
    return 0;
}
```

Output Screenshot

Expected output:

SQUARE(a + 1) = 36
MAX(a, b) = 8
100 rupees = 10000 paise
inline square_int(a + 1) = 36
inline max_int(a, b) = 8

Analysis / Discussion

```
Without parentheses, SQUARE(a + 1) can expand incorrectly because of operator precedence. MAX(i++, j) can evaluate i++ more than once. Do r
```

Result

The macros work for the sample inputs and demonstrate safe parentheses, preprocessor expansion, and the advantage of inline functions.